TravelMind: A Multimodal AI Travel Planning Agent with Tool-Augmented LLM Orchestration

CSCI3280 Project Team

Department of Computer Science and Engineering, The Chinese University of Hong Kong

Abstract

TravelMind is a multimodal travel-planning agent that produces structured day-by-day itineraries from voice or text, grounding every place name, address, and price in live API data rather than LLM generation. Planning decomposes into three sequential, scoped LLM calls (flights, hotels, days), each with a fixed tool allow-list and a Pydantic output contract; this avoids the failure modes of a single end-to-end call. xAI Grok 4.20 was selected as the planner via a 6-model 3-turn benchmark over six diverse prompts (mean 82.1/100, lowest cross-prompt variance of any model tested); the closest premium competitor Claude Sonnet 4.6 collapses on multi-day plans (78 on 3-day $\rightarrow$ 33 on 5-day), while Moonshot Kimi K2 surfaces as the cost-efficient runner-up at 86% of Grok's quality and $3\times$ its score-per-dollar. Server-Sent Events stream a partial itinerary 7.4 s before the final response, converting a 12 s blocking wait into a 4.4 s active wait. The implementation is React 19 / FastAPI, validated by 387 pytest, 57 Vitest, and 54 Playwright browser checks, plus an LLM-judge harness with 31 rubric functions tied to 82 numbered requirements.

1 Introduction

Existing travel tools sit at two extremes. Pure-LLM planners hallucinate plausible-but-wrong places, opening hours, and prices; search engines return raw results the user must aggregate manually. TravelMind bridges both: the LLM *orchestrates* structured itineraries but never *generates* place data—every name, address, latitude, and fare originates in a tool call. The course requires voice input, live API tool calls, and a day-by-day itinerary [1]; Figure 1 shows how those constraints map onto a streaming, tool-augmented React front-end.

2 System Architecture

Three-stage planning pipeline. A single LLM call cannot reliably emit valid flights, hotels, and days together: the context window blows up, mid-stage tool errors corrupt later stages, and the model drifts off the JSON schema. TravelMind splits planning into three scoped `callRole` invocations (Table 1, Figure 2). Each call sees only its own inputs, is restricted to a fixed tool allow-list enforced in `prompts.py:587-607`, and is bounded by `ROLE_MAX_ROUNDS` (`{hotels:3, days:3, day_detail:3}`, `llm.py:93`) before being forced to emit JSON. A fourth `day_detail` role refills a single day on demand; a scoped `replace` role suggests an alternative for one activity using only `search_places`.

Tool layer. Sixteen tools register in `TOOL_DEFINITIONS/TOOL_DISPATCH` (`tools/__init__.py:542`) across four categories: *maps* (`search_places`, `get_place_details`, `get_directions`, `get_weather`, `geocode_city`), *flights* (`search_flights`), *data* (`get_phrasebook`, `get_day_windows`, `search_airports`), and seven *UI-control* tools

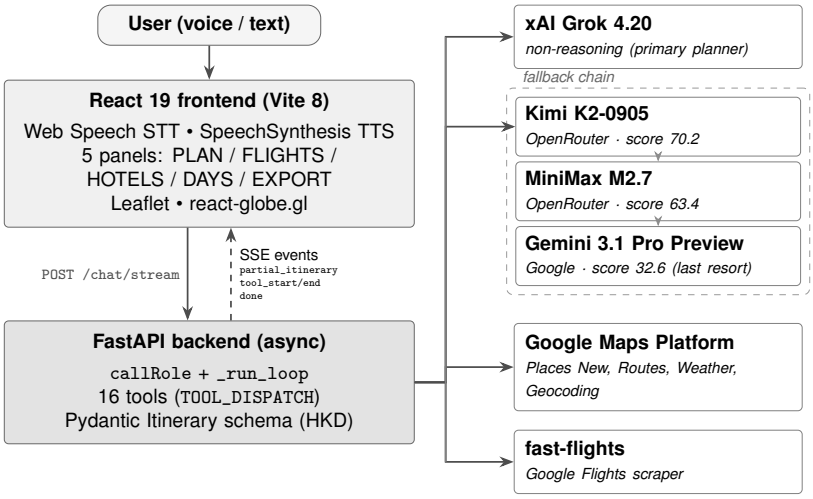

Figure 1: End-to-end architecture. SSE renders panels progressively as `partial_itinerary` events arrive, so flights render at $t \approx 4.4$ s while hotels and days continue streaming until `done` at $t \approx 12$ s.

Stage	Trigger	Tool allow-list	Output
A. Flights	User clicks Start	`search_flights`, `geocode_city`, `get_day_windows`, `get_phrasebook`, `request_input`, `navigate_menu`	`flights[]`, `return_flights[]`, `destination_coords`
B. Hotels	User picks flight	`search_places`, `get_weather`, `navigate_menu`	`hotels[]`, `weather_forecast[]`
C. Days	User picks hotel	`search_places`, `get_directions`, `get_weather`, `navigate_menu`	`days[]`

Table 1: Three scoped LLM stages with tool allow-lists.

(`navigate_menu`, `request_input`, `toggle_setting`, `submit_trip_form`, `pick_flight`, `pick_hotel`, `replace_activity`) that let the LLM advance panels, ask the user a clarifying question, or programmatically pick a flight or hotel mid-flow. Visa data is loaded statically (§4) so no `get_visa` tool exists. Setting `MOCK_TOOLS=1` replaces every tool with a deterministic stub for CI and benchmark reproducibility without API keys.

SSE streaming. `POST /chat/stream` emits 15 distinct event types; four are load-bearing (Table 2), the rest (`thinking`, `token`, `navigate`, `request_input`, `setting_change`, `submit_form`, `pick_flight`, `pick_hotel`, `replace_activity`, `model_fallback`, `error`) drive secondary affordances. A 25 s blank loading state is unacceptable, so the status bar reads "Searching flights…", "Fetching weather…" in real time as each tool fires, and the FLIGHTS panel renders as soon as stage A completes while stage B continues streaming. Empirically `partial_itinerary` arrives 7.4 s before `done` (Figure 3), turning a 12 s blocking wait into a 4.4 s active wait followed by a background "finalising" phase the user can largely ignore.

3 Module Integration

The frontend is a single React 19 component (`App.jsx`, 2,203 lines) that owns all persistent state across nine `localStorage` keys; SSE events from all three stages funnel into one reducer, and panel transitions from hotkeys, tab clicks, and LLM `navigate_menu` calls all route through one `useMenuState` hook. The backend orchestrator (`backend/app/llm.py`) exposes `callRole` (scoped wrapper enforcing allow-lists and `ROLE_MAX_ROUNDS`), `_run_loop` (one tool-call round via `TOOL_DISPATCH`), `_extract_itinerary` (recovers JSON from markdown fences, prose, or trailing-comma output), and `_prune_tool_results` (trims oversized payloads before re-entering context). Pydantic validates every itinerary server-side; `flights[]`, `hotels[]`, and per-day `activity` entries

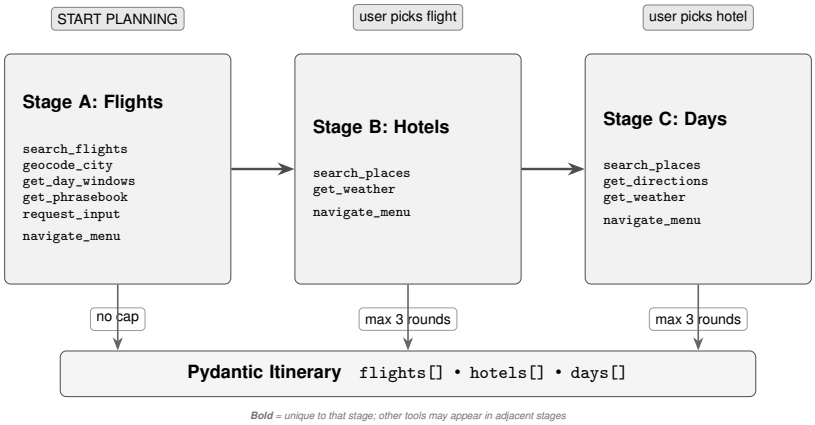

Figure 2: Three-stage planning pipeline: each stage has a fixed tool allow-list and a `ROLE_MAX_ROUNDS` cap; all stages write into the same Pydantic `Itinerary`.

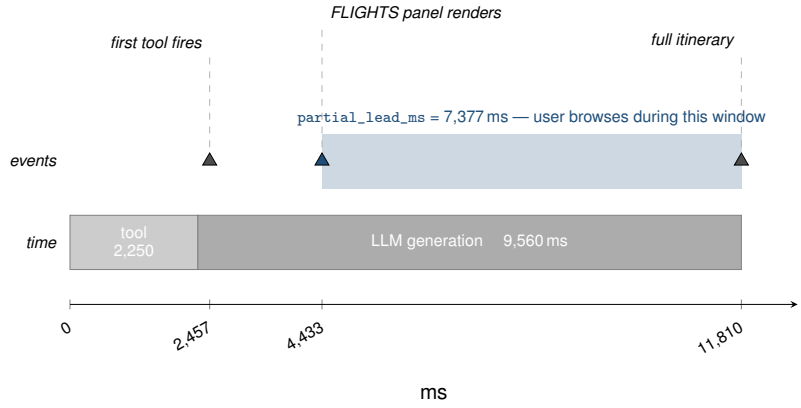

Figure 3: SSE timeline (mean of three live-API runs against `x-ai/grok-4.20`). The shaded span is the 7.4 s window during which the user can browse flight options before `done` arrives. LLM generation (9.6 s) dominates total end-to-end latency; tool execution (2.3 s) is largely parallelised within each stage.

each require a `place_id` so downstream consumers can re-fetch ground truth. Prices are HKD canonical, with client-side static FX driving display—acceptable given $\pm 30\%$ fare confidence bands.

4 Technical Approach and Challenges

LLM selection via benchmarking. `scripts/benchmark-models.py` runs each candidate through the production 3-turn flow (PLAN $\rightarrow$ HOTELS $\rightarrow$ DAYS, role-scoped prompts from `prompts.py:241-407`, deterministic flight/hotel picks injected between turns) over six diverse prompts: 2/3/4/5/7-day trips, four destinations across East Asia / Europe / MENA, plus one train-only edge case (P3 Osaka$\rightarrow$Kyoto). Each cell runs three times under `MOCK_TOOLS=1`; output is scored against rubric v3 (Table 3), 14 weighted criteria totalling 100 points chosen so models discriminate—v2 collapsed every working model to 50/100 because schema/flight-count/phrasebook awarded $15{+}15{+}5$ for any non-broken output.

Results. Two patterns dominate Table 4 and Figure 4. First, quality is bimodal: Grok / Kimi / Minimax cluster at 63–82 while Sonnet / DeepSeek / Gemini Pro cluster at 32–48, with a clean 15-point gap. Second, Sonnet's collapse on long trips is repeatable across destinations (78 on 3-day P1, then 33/34/31/38 on 5-day P2, business-class P4, family-of-3 P5, and 7-day P6), so the failure mode is duration-driven not destination-driven. The score-vs-cost view (Figure 4 left) shows quality and

Event	Payload	Frontend reaction
`tool_start`	`{tool, args}`	status bar shows "calling X…"
`tool_end`	`{tool, summary}`	update status bar
`partial_itinerary`	`{itinerary}`	render panels progressively
`done`	`{itinerary, reply}`	unlock UI, TTS playback

Table 2: Primary SSE events.

#	Criterion	Pts	What it measures
1	Strict schema	10	`place_id`↔lat/lng pairing + monotonic times
2	Flight fidelity / train suppression	15	verbatim copy + `return_options` (P3: omit flights)
3	Hotel count & price diversity	10	5–8 hotels spanning $\geq$3 price levels
4	Day count matches expected	8	`len(days)` $\geq$ `expected_days`
5	`selected_hotel` turn-2 = null	2	turn-2 stage state correct
6	`selected_hotel` turn-3 = picked	2	turn-3 anchors on the right hotel
7	Activity density	12	avg. $\geq$4 real activities per day
8	Day-1 anchor	5	first two activities = arrival airport $\rightarrow$ hotel
9	Last-day anchor	5	final activity = departure airport
10	Activity-kind diversity	6	penalty for 3+ same kind in one day
11	Description grounding	6	every `place_id` has a non-empty matching description
12	Directions populated	6	`transport_to_next` on every consecutive pair
13	Phrasebook fidelity	5	non-empty phrases (P3 exempt: domestic)
14	Tool-call efficiency	8	within 2$\times$ expected $\rightarrow$ full; 3$\times$ $\rightarrow$ half

Table 3: Benchmark rubric v3: 14 criteria, 100 points total (`score_v3` in `benchmark-models.py:614`).

cost-efficiency are orthogonal: cost varies 19$\times$ across the cohort (Kimi 2074 score/$ vs. Sonnet 107), and the cheapest model (DeepSeek $0.023/run) outranks two premium models on score-per-dollar despite landing in the lower quality cluster. The score-vs-time view (Figure 4 right) shows latency is similarly uncorrelated with quality—Grok runs faster than Sonnet despite scoring 34 points higher.

Model	P1	P2	P3	P4	P5	P6	Mean	$/run	score/$
x-ai/grok-4.20	**89**	**88**	83	74	**83**	75	**82.1**	0.129	635
moonshotai/kimi-k2-0905	58	63	79	70	71	**81**	70.2	0.034	**2074**
minimax/minimax-m2.7	74	62	65	54	53	73	63.4	0.040	1573
anthropic/claude-sonnet-4.6	78	33	77	34	31	38	48.3	0.451	107
deepseek/deepseek-v3.2	41	25	41	29	47	44	37.9	0.023	1637
google/gemini-3.1-pro-preview	44	27	35	38	22	30	32.6	0.151	216

Table 4: 6-model 3-turn benchmark, six prompts $\times$ three runs each (mean score / 100). `$/run` averages all 18 cells per model; `score/$` is mean$\div$`$/run`. P3 is the train-only Osaka$\rightarrow$Kyoto prompt; P6 is the 7-day Marrakech stress test.

Decision. Grok 4.20 is the only model with cross-prompt mean $\geq$80 *and* the lowest single-prompt variance (74–89), so it is the default (`LLM_MODEL` in `config.py`). Kimi K2-0905 retains 86% of Grok's quality at 26% of its per-run cost; `config.py:FALLBACK_CHAIN` walks Kimi $\rightarrow$ MiniMax M2.7 $\rightarrow$ Gemini 3.1 Pro Preview on outage, ranked by the same v3 score, emitting a `model_fallback` SSE on each step.

External data without paid APIs. Real flight APIs (Amadeus, Skyscanner) require business contracts and charge per call, so TravelMind uses `fast-flights` [3] backed by `primp`, a TLS-fingerprint-mimicking HTTP client; a monkey-patch on its internal `fetch` captures raw HTML so a second pass mines layover stop-city metadata the upstream parser drops, with a deterministic price model filling in fares when scraping is geo-blocked. Visa requirements (199 passports $\times$ 200+ destinations) come from the MIT-licensed `ilyankou/passport-index-data` [4] (Feb 2026) plus a hand-verified Hong-Kong overlay; statuses are normalised to a six-state enum and surfaced via a `VisaAlertBanner` on the FLIGHTS panel before the user picks a fare.

Prompt engineering. Four techniques together make JSON output reliable. **(i)** Every prompt rule traces to one of the 82 `R-*` IDs in `docs/llm-spec.md`; rubric functions (§5) cite the same IDs, so

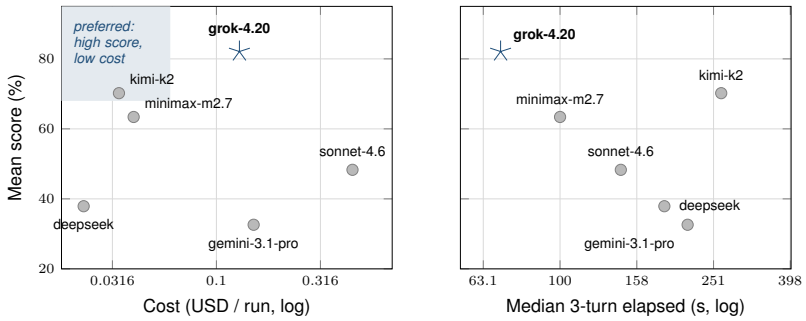

Figure 4: 6-model bench, two-axis view. *Left:* mean score vs. cost per run (USD, log scale)—Kimi K2-0905 is Pareto-best on cost-efficiency at 3× Grok's score-per-dollar. *Right:* mean score vs. median 3-turn elapsed time (s, log scale). Grok 4.20 (highlighted) is deployed; the cost/quality and latency/quality axes carry independent information.

a failing rubric points at exactly one rule. **(ii)** `prompts.py` embeds a ∼120-line concrete example itinerary; models follow examples more reliably than abstract schema text. **(iii)** Scoped allow-lists (Table 1) reject out-of-stage tool calls before they hit the network. **(iv)** `_extract_itinerary()` handles JSON inside markdown fences, in prose, or with trailing commas; `_prune_tool_results()` keeps the context window under budget across all three rounds.

5 Extended Features and Evaluation

Table 5 summarises features beyond the MVP and the test surface that guards them. The 31 rubric functions in `backend/app/evals/rubrics.py` are each tied to one `R-*` ID from `docs/llm-spec.md` (82 IDs total) and judged by Gemini 2.5 Flash; a Grok-judges-Grok pilot scored itself leniently, so a cross-vendor judge was substituted.

Feature	**Evaluation surface**
199-passport visa-alert banner (§4)	387 pytest cases over 17 modules
3-D globe arcing origin→destination [6]	57 Vitest unit tests
Per-hotel and per-day Leaflet maps [5]	5 Playwright E2E specs (54 browser checks)
Favourites (★, F key); 12-item checklist (L key)	plan, keyboard nav, overlays, persistence, 1024×600 viewport
HKD currency switching (USD/EUR/JPY/GBP/CNY, S key)	31 `check_R_*` rubric functions
KML + PDF export via WeasyPrint [7]	each tied to one R-ID; cross-vendor LLM judge (Gemini 2.5 Flash)
Scoped `replace` role; plan history (H, drag-drop)	82-ID requirement spec (`docs/llm-spec.md`)
Live service-status overlay (C key)	`MOCK_TOOLS=1` fixture mode for deterministic CI

Table 5: Extended features (left) and evaluation surface (right).

References

[1] CSCI3280 Spring 2026, *Project Specification: Multimedia Travel Planning Agent*, CUHK, 2026.

[2] OpenAI, *Function Calling Guide*, `https://platform.openai.com/docs/guides/function-calling`.

[3] *fast-flights* (Google Flights scraper), `https://github.com/AWeirdDev/flights`.

[4] *ilyankou/passport-index-data*, MIT License, `https://github.com/ilyankou/passport-index-data`.

[5] *Leaflet.js*, `https://leafletjs.com`.

[6] V. Asturiano, *react-globe.gl*, `https://github.com/vasturiano/react-globe.gl`.

[7] *WeasyPrint*, `https://weasyprint.org`.

[8] React Team, *React 19 Release Notes*, 2024, `https://react.dev/blog/2024/12/05/react-19`.

[9] xAI, *Grok Model Card*, `https://x.ai/blog/grok`.

[10] Google, *Google Maps Platform: Places (New), Routes, Weather APIs*, `https://developers.google.com/maps`.

Appendix A. Team Contributions

Area	Member	Contribution
Core System Architecture & Full-Stack	Core-system lead	**LLM Orchestration:** Developed 3-stage `callRole` pipeline with v3-ranked `FALLBACK_CHAIN` (Kimi/MiniMax/Gemini); implemented `model_fallback` SSEs and Pydantic grounding schemas. **Backend & Tooling:** Built FastAPI async infra; integrated 16 tools including Google Maps (Places/Routes), `fast-flights` with TLS-fingerprinting, and custom Visa/IATA databases. **Frontend Engineering:** Developed React 19 state-machine with 15 SSE handlers; integrated Leaflet/3D-globe visualizations and dynamic KML/PDF export engines. **Evaluation & DevOps:** Established 14-criterion 6-model benchmark; authored 400+ unit/E2E tests (Pytest/Playwright); managed CI/CD and GitHub repository. Provisioned and managed a dedicated Linux staging server; configured secure remote access via Tailscale for team-wide testing and deployment. **Reporting:** Primary author of the final report and all technical documentation (`llm-spec.md`, `design.md`); produced all TikZ/PGFPlots figures.
Project Management & Quality Assurance	Project / QA lead	**Project Lifecycle:** Managed team milestones, schedule enforcement, and team coordination. **Documentation:** Managed bug-tracking resolution logs and finalized report formatting/alignment.
Validation & System Testing	Test lead	**Functional QA:** Executed end-to-end testing of Activity Plan logic; validated mapping accuracy for hotel-to-destination modules. **Data Integrity:** Tracked flight/itinerary synchronization issues in the central tracking document.
User Journey & Presentation	UX / presentation lead	**UX Testing:** Performed manual validation of all 5 UI panels and state transition edge cases. **Dissemination:** Coordinated presentation narrative.

This project was developed with assistance from Claude Code.